

Tribhuvan University
Faculty of Humanities and Social Sciences

[Insert Tribhuvan University logo here]

A PROJECT REPORT ON
"STAYEASE: ONLINE HOTEL BOOKING AND
MANAGEMENT SYSTEM"

Submitted to
Department of Computer Application
[COLLEGE NAME]

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by
[YOUR FULL NAME] ([TU Roll No.]

[Month Year]

Under the Supervision of
[Supervisor Name]

Tribhuvan University
Faculty of Humanities and Social Sciences

A PROJECT REPORT ON
"STAYEASE: ONLINE HOTEL BOOKING AND
MANAGEMENT SYSTEM"

Submitted to
Department of Computer Application
[COLLEGE NAME]

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by
[YOUR FULL NAME] ([TU Roll No.]

[Month Year]

Under the Supervision of
[Supervisor Name]

Tribhuvan University
Faculty of Humanities and Social Sciences

[COLLEGE NAME]

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **[YOUR FULL NAME]** entitled "STAYEASE: ONLINE HOTEL BOOKING AND MANAGEMENT SYSTEM" in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

SIGNATURE

[Supervisor Name]

SUPERVISOR

[Academic Designation]

Department of Computer Application

[COLLEGE NAME], [Full Address]

Tribhuvan University
Faculty of Humanities and Social Sciences

[COLLEGE NAME]

LETTER OF APPROVAL

This is to certify that this project prepared by **[YOUR FULL NAME]** entitled "STAYEASE: ONLINE HOTEL BOOKING AND MANAGEMENT SYSTEM" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

.....
[Supervisor Name]
Supervisor
[Academic Designation]
Department of Computer Application
[COLLEGE NAME]

.....
[Coordinator / HOD Name]
Coordinator / HOD
[Academic Designation]
Department of Computer Application
[COLLEGE NAME]

.....
[Internal Examiner Name]
Internal Examiner
Department of Computer Application
[COLLEGE NAME]

.....
[External Examiner Name]
External Examiner
Tribhuvan University
Department of Computer Application
[COLLEGE NAME]

ABSTRACT

StayEase is a web-based hotel room booking and management system developed to replace the manual, register-based reservation process still used by many small and medium hotels in Nepal. In such hotels, bookings are taken by phone or at the front desk and recorded on paper or in spreadsheets, which leads to double-booked rooms, lost reservations, slow response to guests, and no reliable picture of occupancy or revenue.

The system gives customers a single online portal where they can register, search rooms by check-in date, check-out date, number of guests, price and room type, view room details and photos, place a booking, pay through a demo payment step, and view or cancel their own bookings. Every booking receives a unique booking code and its total cost is calculated automatically from the nightly price and the number of nights. Administrators use a separate panel to manage room types and individual rooms with images and amenities, change room status for maintenance, confirm bookings, record check-in and check-out, update payment status, view customers, and follow key figures such as total bookings, occupancy and revenue on a dashboard.

The central problem the system solves is double booking. Before a booking is saved, the system checks for any active booking whose dates overlap the requested stay, and it repeats this check inside a database transaction that locks the room row, so two customers cannot reserve the same room for the same night even when they submit at the same moment. Security is handled through bcrypt password hashing, role-based access control, CSRF tokens, prepared statements against SQL injection, output escaping against cross-site scripting, login rate limiting and validated file uploads.

The system is built with HTML5, CSS3 and JavaScript on the front end, PHP 8 on the server, and a MySQL (MariaDB) database, and it runs on the XAMPP stack. It was developed using an iterative Agile approach and verified with unit and system test cases.

Keywords: Hotel Booking System, Room Reservation, Availability Checking, Double-Booking Prevention, Role-Based Access Control, Web Application, PHP, MySQL, XAMPP

ACKNOWLEDGEMENT

The successful completion of this project, "StayEase: Online Hotel Booking and Management System", was possible because of the guidance, support and encouragement of many people, and I would like to express my sincere gratitude to all of them.

I am deeply grateful to my project supervisor, [Supervisor Name], for continuous guidance, constructive feedback and encouragement at every stage of this project. The suggestions received during the proposal, mid-term and final phases shaped both the design and the quality of this work.

I would also like to thank [Coordinator / HOD Name], Coordinator of the Department of Computer Application, and all faculty members of [COLLEGE NAME] for their academic support, valuable advice, and for providing the laboratory facilities and environment needed to carry out this project.

I am thankful to Tribhuvan University, Faculty of Humanities and Social Sciences, for including Project I (CACS256) in the BCA curriculum, which gave me the opportunity to apply classroom knowledge to a real-world problem.

Finally, I am thankful to my family and friends for their constant support, motivation and patience throughout the development of this project.

[YOUR FULL NAME]

[TU Roll No.]

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION.....	i
LETTER OF APPROVAL	ii
ABSTRACT.....	iii
ACKNOWLEDGEMENT	iv
LIST OF ABBREVIATIONS.....	vii
LIST OF FIGURES	viii
LIST OF TABLES.....	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction.....	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.4.1 Scope.....	2
1.4.2 Limitations	3
1.5 Report Organization.....	3
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	5
2.1 Background Study.....	5
2.2 Literature Review.....	6
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	8
3.1 System Analysis.....	8
3.1.1 Requirement Analysis.....	8
3.1.2 Feasibility Analysis.....	10
3.1.3 Methodology Used.....	11
3.1.4 Data Modelling (ER Diagram).....	13
3.1.5 Process Modelling (DFD).....	14

3.2 System Design	16
3.2.1 Architectural Design	16
3.2.2 Database Schema Design	17
3.2.3 Interface Design	19
3.2.4 Physical DFD	21
CHAPTER 4: IMPLEMENTATION AND TESTING	23
4.1 Implementation	23
4.1.1 Tools Used	23
4.1.2 Implementation Details of Modules.....	24
4.2 Testing.....	26
4.2.1 Test Cases for Unit Testing.....	26
4.2.2 Test Cases for System Testing.....	27
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS	29
5.1 Lesson Learnt / Outcome	29
5.2 Conclusion	29
5.3 Future Recommendations	30
REFERENCES	31
APPENDICES	32
Appendix A: Screenshots.....	32
Appendix B: Source Code Excerpt	36
Appendix C: Supervisor Visit Log Sheet.....	37

LIST OF ABBREVIATIONS

AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
CASE	Computer Aided Software Engineering
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDOR	Insecure Direct Object Reference
JSON	JavaScript Object Notation
NPR	Nepalese Rupee
OTA	Online Travel Agency
PDO	PHP Data Objects
PHP	PHP: Hypertext Preprocessor
PMS	Property Management System
RBAC	Role-Based Access Control
SQL	Structured Query Language
UI	User Interface
VS Code	Visual Studio Code
XAMPP	Cross-platform, Apache, MariaDB, PHP and Perl
XSS	Cross-Site Scripting

LIST OF FIGURES

Figure 3.1: Use Case Diagram of StayEase	9
Figure 3.2: Gantt Chart	11
Figure 3.3: Agile Development Cycle	12
Figure 3.4: ER Diagram of StayEase	13
Figure 3.5: Context Diagram (Level 0 DFD).....	14
Figure 3.6: Level 1 DFD	15
Figure 3.7: Level 2 DFD of Process 3.0 (Book Room)	16
Figure 3.8: Three-Tier Architecture of StayEase.....	17
Figure 3.9: Database Schema.....	18
Figure 3.10: Home Page with Room Search.....	19
Figure 3.11: Room Details and Booking Form.....	20
Figure 3.12: Admin Dashboard.....	20
Figure 3.13: Admin Booking Management	21
Figure 3.14: Physical DFD	22

LIST OF TABLES

Table 3.1: Project Schedule	10
Table 3.2: Structure of the bookings Table.....	18
Table 4.1: Unit Test Cases for Registration.....	26
Table 4.2: Unit Test Cases for Login.....	26
Table 4.3: Unit Test Cases for Room Search and Booking	26
Table 4.4: Unit Test Cases for Payment and Cancellation	27
Table 4.5: Unit Test Cases for Admin Room Management.....	27
Table 4.6: System Test Case for the Complete Booking Journey	27
Table C.1: Supervisor Visit Log Sheet	37

CHAPTER 1: INTRODUCTION

1.1 Introduction

StayEase is a web-based Online Hotel Booking and Management System that allows hotel guests to find and reserve rooms over the internet and allows hotel staff to manage rooms, bookings and payments from a single administrative panel. The system is designed around two roles. Customers can register, search for rooms that are free for their chosen dates, see each room's price, capacity, amenities and photos, book a room, pay, and follow the status of their bookings. Administrators can create room categories such as Deluxe or Suite, add individual rooms with room numbers and nightly prices, upload photos, place rooms under maintenance, confirm or cancel bookings, record check-in and check-out, update payment status, and view a dashboard of rooms, bookings, customers, occupancy and revenue.

The core of the system is its availability engine. A room is shown as available only when it is in the "available" state and has no active booking (pending, confirmed or checked-in) whose dates overlap the requested stay. The same rule is enforced again inside a database transaction at the moment of booking, with the room row locked, so that two guests who submit at the same time cannot both reserve the same room. Each booking receives a unique booking code, and the total amount is calculated automatically as the nightly price multiplied by the number of nights.

The application is built with HTML5, CSS3 and JavaScript for the interface, PHP 8 for server-side logic and MySQL (MariaDB) as the database, and it runs on the XAMPP stack. Prices are shown in Nepalese Rupees (NPR), and the payment step is implemented in demo mode so that it can later be connected to local gateways such as eSewa or Khalti.

1.2 Problem Statement

Many small and medium hotels, guest houses and resorts still take reservations by telephone, walk-in or social media messages and record them in a paper register or a spreadsheet. This way of working causes several problems:

- **Double booking:** Two staff members, or a phone booking and a walk-in, can assign the same room for overlapping nights because there is no single, real-time record of availability.

- **No self-service for guests:** Guests cannot see which rooms are free, their prices or their facilities without contacting the hotel, and they cannot book outside office hours.
- **Slow and error-prone calculations:** The number of nights and the total bill are calculated by hand, which leads to mistakes and disputes.
- **Poor record keeping:** Paper records are easily lost or damaged, are hard to search, and do not show the history of a guest's bookings or payments.
- **No management overview:** The owner has no quick way to see occupancy, pending bookings, today's check-ins or total revenue.

There is therefore a need for a simple, secure and affordable web application that keeps one authoritative record of rooms and bookings, lets guests book online, and gives the hotel a clear view of its operations.

1.3 Objectives

The main objective of this project is to develop a secure web-based hotel booking and management system. The specific objectives are:

- To allow customers to search available rooms by dates, guests, price and room type, and to book and pay for a room online.
- To prevent double booking through date-overlap checking and transaction-level locking of rooms.
- To provide an administrative panel for managing room types, rooms, bookings, payments and customers, with a statistics dashboard.

1.4 Scope and Limitation

1.4.1 Scope

- Customers can register, log in, search rooms, view room details, book a room, pay through the demo payment step, view their booking history and cancel eligible bookings.
- Administrators can manage room types and rooms, including images, gallery photos, amenities, prices and status (available, unavailable, maintenance).

- Administrators can view and filter all bookings, update booking status through its full life cycle (pending, confirmed, checked-in, checked-out, cancelled) and update payment status.
- The dashboard shows total rooms, bookings, customers, revenue, occupancy rate, pending bookings and today's check-ins.
- A JSON availability endpoint returns the free rooms for given dates and guests, which can be reused by other clients in the future.
- The system is suitable for a single small or medium hotel and runs in any modern web browser.

1.4.2 Limitations

- Online payment works in demo mode only; no real money is transferred and no live payment gateway such as eSewa or Khalti is integrated yet.
- The system does not send email or SMS notifications for booking confirmation or cancellation.
- It supports a single hotel property; multiple branches or hotel chains are not supported.
- There is no separate role for front-desk staff, housekeeping or accounts; all staff functions are performed through the admin role.
- Guest reviews, ratings, discount coupons, seasonal pricing and refunds are not implemented.
- The system has been tested on a local XAMPP server with a small data set and has not been load-tested for heavy traffic.

1.5 Report Organization

This report is organized into five chapters:

Chapter 1: Introduction introduces the project, the problem it addresses, its objectives, and its scope and limitations.

Chapter 2: Background Study and Literature Review explains the concepts behind online hotel reservation and reviews existing systems and related studies.

Chapter 3: System Analysis and Design presents the functional and non-functional requirements, the feasibility analysis, the development methodology, the ER diagram, the data flow diagrams, and the architectural, database and interface design.

Chapter 4: Implementation and Testing describes the tools used, the implementation of each module, and the unit and system test cases.

Chapter 5: Conclusion and Future Recommendations summarizes the lessons learnt, the outcome of the project and possible future enhancements.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Hotel reservation is the process of holding a specific room, or a room of a given category, for a guest for a given range of nights. The key rule that any reservation system must enforce is that a single room cannot be occupied by two guests on the same night. In StayEase a booking is described by a room, a check-in date and a check-out date, where the check-out date is exclusive: a guest who checks out on the 5th frees the room for a new guest arriving on the 5th. Two bookings for the same room therefore conflict only when the existing check-in is earlier than the new check-out and the existing check-out is later than the new check-in. This half-open interval test is the basis of the availability engine described in Chapter 3.

The hospitality industry was among the earliest adopters of information technology. Large chains have long used central reservation systems and property management systems (PMS), and the growth of the internet moved booking directly to the guest through hotel websites and online travel agencies (OTAs) such as Booking.com and Agoda [1], [2]. Buhalis and Law observed that the internet changed how travellers search for and buy accommodation and that hotels which do not offer online booking lose visibility and business [1]. For small and medium hotels, however, commercial PMS and OTA commissions are often costly, which leaves a gap for simple, low-cost direct booking systems.

A web-based booking system follows a client-server model. The guest uses a browser (the client) to send HTTP requests to a web server, server-side code processes the request and talks to a relational database, and an HTML page is returned [3]. Because many guests may try to book the same room at once, the database must guarantee consistency. Relational databases such as MySQL provide transactions with the ACID properties (atomicity, consistency, isolation and durability) and row-level locks such as `SELECT ... FOR UPDATE`, which make it possible to check availability and insert a booking as one indivisible step [4], [5].

Because a booking system stores personal data and accepts payments, security is a central concern. The OWASP Top 10 lists injection, broken access control, cryptographic

failures and security misconfiguration among the most critical web application risks [6]. Standard countermeasures include parameterized queries, strong password hashing, anti-CSRF tokens, output encoding and role-based access control (RBAC), where each user can reach only the functions allowed for their role [6]-[8]. StayEase applies each of these measures.

2.2 Literature Review

Large OTAs such as Booking.com and Agoda offer room search by date and guests, photos, reviews, instant confirmation and online payment. They serve as the reference model for the customer side of StayEase, particularly the search-then-book flow and the unique booking reference. Their main drawback for small hotels is commission on every booking and limited control over the guest relationship, which is why many hotels also want a direct booking channel [2].

O'Connor and Frew studied hotel electronic distribution and found that direct channels such as the hotel's own website were expected to grow in importance because they reduce distribution costs and give hotels more control over pricing and inventory [2]. This supports the idea of a lightweight system that a single hotel can host itself.

Commercial PMS products, such as eZee Absolute which is widely used by hotels in Nepal, combine front-desk operations, room status, billing and channel management. They show the importance of a clear booking life cycle (reserved, confirmed, checked-in, checked-out, cancelled) and of room status for maintenance. StayEase adopts the same life cycle and room status model in a simplified form suitable for an academic project.

Many student and academic hotel reservation projects built with PHP and MySQL implement registration, room listing and booking, but they commonly check availability with a single SELECT before an INSERT and do not handle two users booking at the same moment, which can still result in double bookings. Sommerville and Pressman both emphasize that such non-functional requirements, including reliability, security and concurrency, must be designed in rather than added later [3], [9]. StayEase addresses this gap by re-checking availability inside a transaction with a locked room row and by rejecting duplicate submissions made within five minutes.

From this review, the features selected for StayEase are online search by dates and guests, clear pricing with automatic total calculation, a unique booking code, a full booking

status life cycle, room maintenance status, an administrative dashboard, and strong protection against double booking and common web attacks.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis was carried out by identifying requirements, studying feasibility, choosing a development methodology, and modelling data and processes, as described below.

3.1.1 Requirement Analysis

i. Functional Requirements

The functional requirements of StayEase are:

- **User registration and login:** Customers register with name, email, 10-digit phone number and a strong password, then log in and log out. Administrators log in to a separate panel.
- **Room search:** Anyone can list room types and rooms and filter them by check-in date, check-out date, guests, maximum price and room type. Only rooms free for the whole stay are shown.
- **Room details:** Logged-in customers see a room's photos, gallery, amenities, capacity, nightly price and the calculated total for the selected nights.
- **Booking:** Customers book a room with contact details and an optional special request. The system validates input, checks availability, generates a booking code and saves the booking as pending.
- **Payment:** Customers pay for a pending booking through the demo payment page, after which the booking becomes confirmed.
- **My bookings and cancellation:** Customers view their bookings and cancel a booking while it is pending or confirmed.
- **Room type and room management:** Administrators add, edit, enable, disable and delete room types and rooms, upload images and set room status.
- **Booking management:** Administrators view and filter all bookings and update booking status and payment status.
- **Customer list and dashboard:** Administrators view registered customers and a dashboard of key statistics and recent bookings.

The use case diagram in Figure 3.1 shows these functions and the actors who use them.

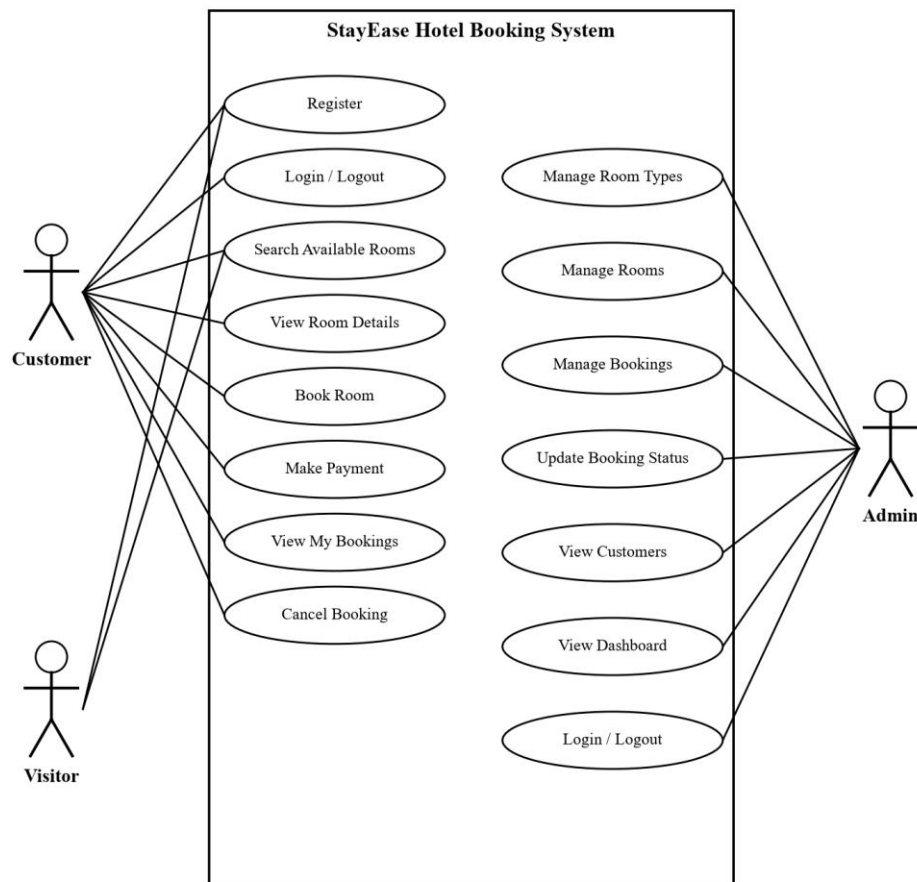


Figure 3.1: Use Case Diagram of StayEase

ii. Non-Functional Requirements

- **Security:** Passwords are hashed with bcrypt, all queries use prepared statements, every form carries a CSRF token, output is escaped, login is limited to five attempts per 15 minutes, and admin pages require the admin role.
- **Reliability and consistency:** A room can never be booked twice for overlapping nights, even under concurrent requests, because the final check and insert run in one transaction with a row lock.
- **Usability:** The interface is clean, responsive and works on desktop, tablet and mobile browsers, with clear validation messages and toast notifications.
- **Performance:** Pages load quickly on a local server, and the bookings table is indexed on room and dates to keep availability queries fast.

- **Availability and portability:** The system runs on any machine with Apache, PHP 8 and MySQL and is accessed from any modern browser.
- **Maintainability:** Common functions such as database access, escaping, CSRF and availability checking are kept in one configuration file and reused across pages.

3.1.2 Feasibility Analysis

A feasibility study was carried out before development to confirm that the system could be built and used within the available time and resources.

i. Technical Feasibility

StayEase uses mature, widely available and free technologies: HTML5, CSS3, JavaScript, PHP 8 and MySQL, all packaged in XAMPP. These technologies were studied during the BCA course, run on an ordinary laptop, and are supported by almost every web hosting provider. No special hardware is required. The project is therefore technically feasible.

ii. Operational Feasibility

The system follows the same steps that guests already know from popular booking websites: choose dates, pick a room, book and pay. The admin panel uses simple forms, tables, filters and status buttons that hotel staff can learn quickly. Because it removes manual calculation and register keeping, staff have a clear reason to adopt it. The system is operationally feasible.

iii. Economic Feasibility

All development tools and runtime software are free and open source, so there is no licensing cost. The only costs of real deployment would be a domain name and shared hosting, which are low compared with OTA commissions or commercial PMS subscriptions. The system is economically feasible.

iv. Schedule Feasibility

The project was planned over the fourth semester and divided into phases as shown in Table 3.1 and the Gantt chart in Figure 3.2. The planned work fitted within the semester, so the project is feasible in terms of schedule.

Table 3.1: Project Schedule

Week	Task	Description
1-2	Requirement Analysis	Study of manual booking, existing systems, and

Week	Task	Description
		collection of requirements.
3-4	System Design	Use case, ER diagram, DFDs, database schema and interface design.
4-6	Frontend Development	Home, rooms, room details, login, register and my bookings pages.
5-8	Backend Development	Authentication, availability logic, booking, payment and cancellation.
7-9	Admin Panel	Dashboard, room types, rooms, bookings and customers modules.
9-10	Testing	Unit and system testing, bug fixing and security hardening.
10-12	Documentation	Preparation of the project report and final review.

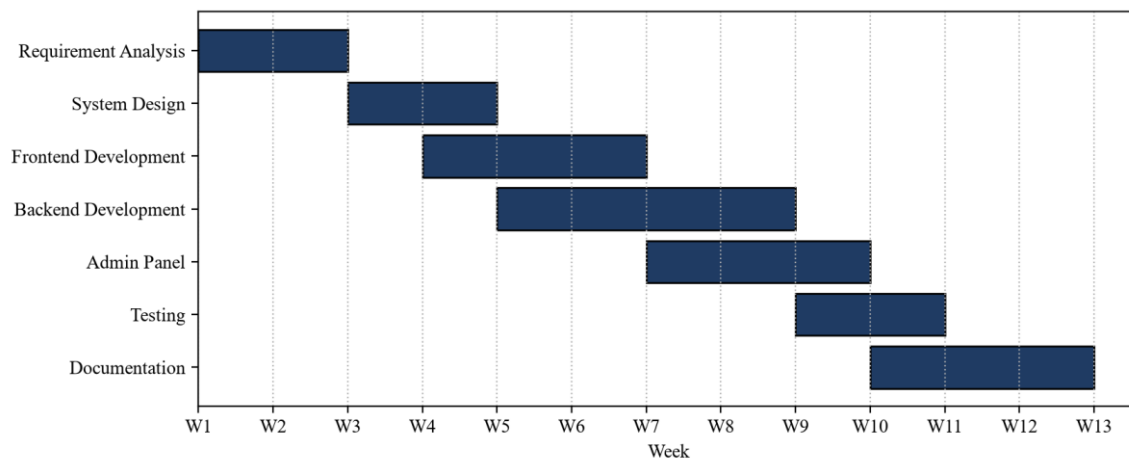


Figure 3.2: Gantt Chart

3.1.3 Methodology Used

The Agile methodology was used to develop StayEase. The system was built in short iterations, and each iteration delivered a working part of the system that was tested and reviewed before the next iteration began. This suited the project because the requirements became clearer as the system was used; for example, the need for room maintenance status, duplicate booking protection and payment status updates by the admin was identified during reviews and added in later iterations.

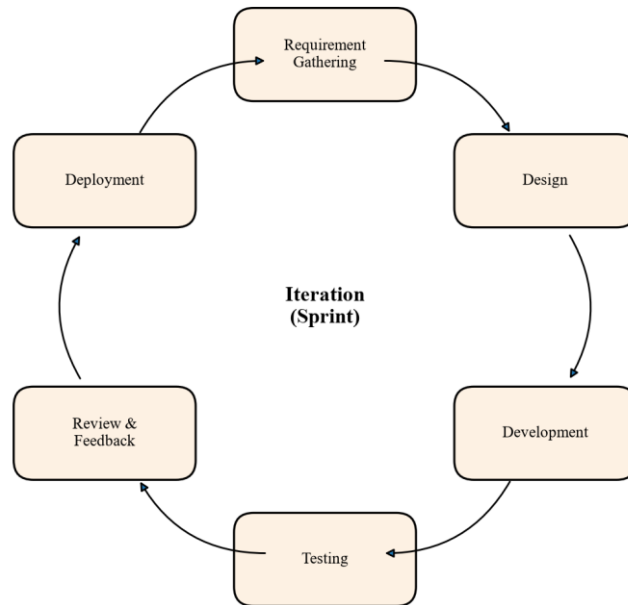


Figure 3.3: Agile Development Cycle

The Agile phases were applied to StayEase as follows:

- **Requirement Gathering:** Features were listed from the study of manual hotel booking and existing booking websites, and then prioritized.
- **Design:** For each iteration, the related tables, pages and data flows were designed.
- **Development:** Iteration 1 built authentication and the database; iteration 2 built room listing, search and availability; iteration 3 built booking, payment and cancellation; iteration 4 built the admin panel; iteration 5 added security hardening and UI improvements.
- **Testing:** Each feature was tested with valid and invalid input before being accepted.
- **Review and Feedback:** Working features were reviewed with the supervisor, and the feedback was added to the next iteration.
- **Deployment:** Each completed iteration was deployed on the local XAMPP server.

3.1.4 Data Modelling (ER Diagram)

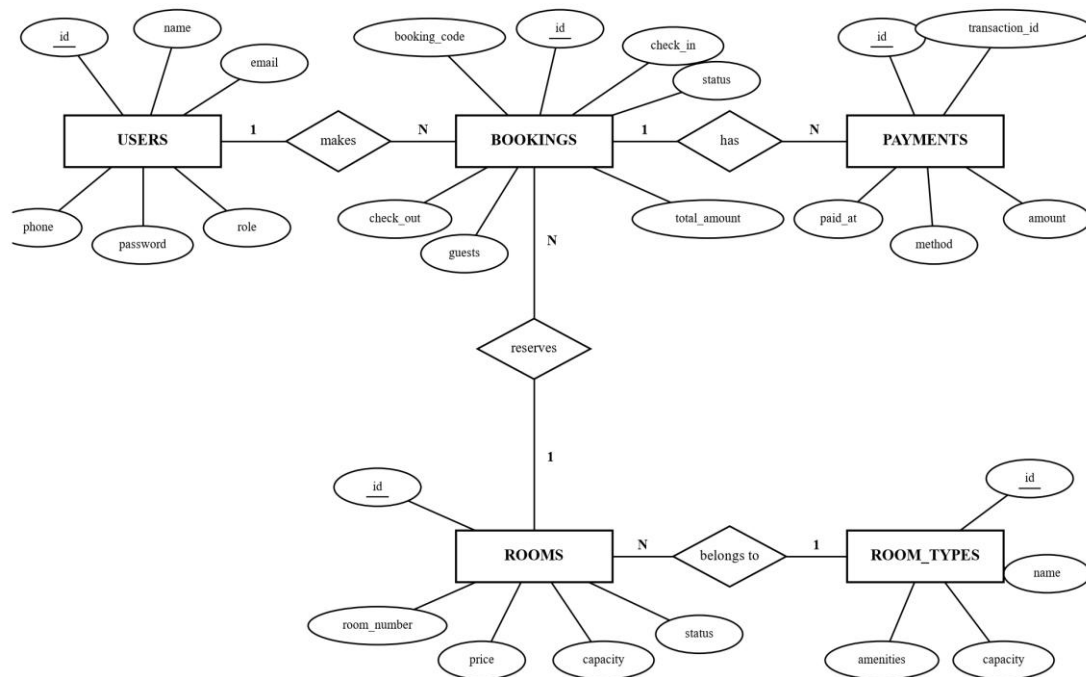


Figure 3.4: ER Diagram of StayEase

The ER diagram in Figure 3.4 contains five entities. **USERS** stores both customers and administrators, distinguished by the role attribute. A user makes many **BOOKINGS**, and each booking belongs to exactly one user (1:N). Each booking reserves exactly one room, while one **ROOM** can appear in many bookings over time (N:1). Each room belongs to one **ROOM_TYPE**, and a room type groups many rooms (N:1). A booking has zero or more **PAYMENTS**, for example a demo online payment or a cash payment recorded by the admin (1:N). Booking status and payment status are stored as enumerated attributes so that the booking life cycle can be tracked.

3.1.5 Process Modelling (DFD)

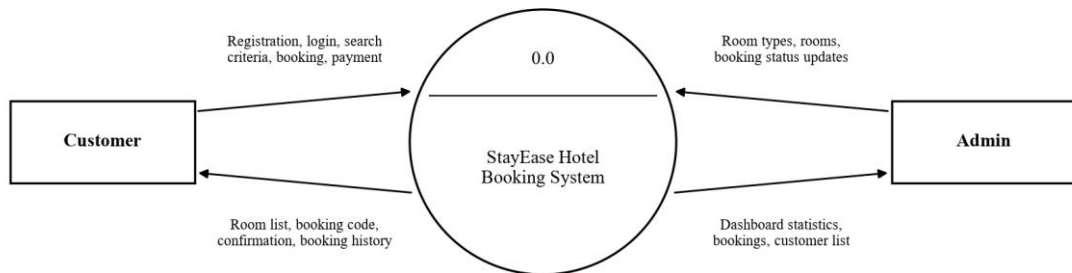


Figure 3.5: Context Diagram (Level 0 DFD)

The context diagram in Figure 3.5 shows StayEase as a single process with two external entities. The customer sends registration and login details, search criteria, booking requests and payment, and receives room lists, booking codes, confirmations and booking history. The admin sends room type and room data and booking status updates, and receives dashboard statistics, booking lists and customer lists.

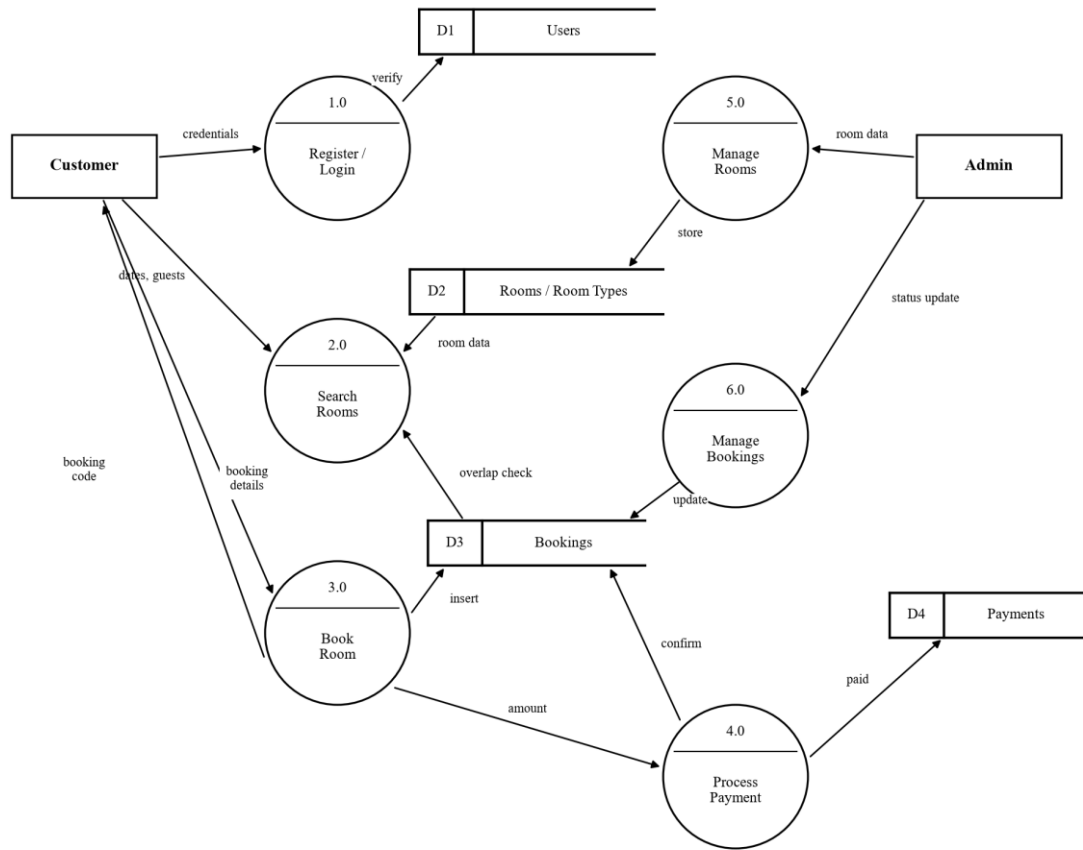


Figure 3.6: Level 1 DFD

The Level 1 DFD in Figure 3.6 breaks the system into six processes. Register/Login (1.0) verifies credentials against the Users store (D1). Search Rooms (2.0) reads the Rooms and Room Types store (D2) and checks the Bookings store (D3) for overlapping stays. Book Room (3.0) saves a new booking in D3 and returns the booking code. Process Payment (4.0) records the payment in D4 and confirms the booking in D3. On the admin side, Manage Rooms (5.0) writes to D2 and Manage Bookings (6.0) updates booking and payment status.

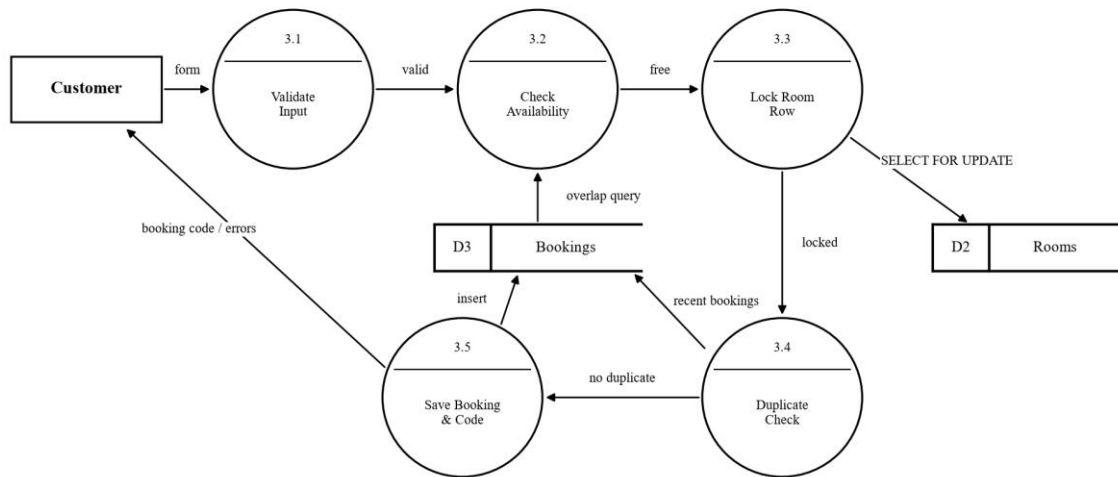


Figure 3.7: Level 2 DFD of Process 3.0 (Book Room)

Figure 3.7 expands the Book Room process, which is the most critical part of the system. The booking form is first validated (3.1): dates must not be in the past, the stay must be between 1 and 30 nights, guests must not exceed room capacity, and contact details must be valid. Availability is then checked against active bookings (3.2). Inside a transaction, the room row is locked with `SELECT ... FOR UPDATE` and availability is checked again (3.3). A duplicate check (3.4) rejects the same booking made by the same user within the last five minutes, which prevents double clicks. Finally, the booking is saved with a unique code and a calculated total (3.5), and the code or an error message is returned to the customer.

3.2 System Design

The system design of StayEase consists of the architectural design, database schema design, interface design and physical DFD.

3.2.1 Architectural Design

StayEase follows a three-tier client-server architecture, shown in Figure 3.8.

- **Presentation tier:** HTML5 pages styled with CSS3, with JavaScript for date validation, price preview, image galleries, modals and toast notifications. It includes public pages, customer pages and the admin panel.

- **Application tier:** PHP 8 scripts running on Apache. They handle sessions, authentication, CSRF protection, rate limiting, validation, availability checking, booking transactions, payments and file uploads. A small JSON API returns available rooms.
- **Data tier:** A MySQL (MariaDB) database named `hotel_booking` holding the `users`, `room_types`, `rooms`, `bookings` and `payments` tables, accessed only through PDO prepared statements.

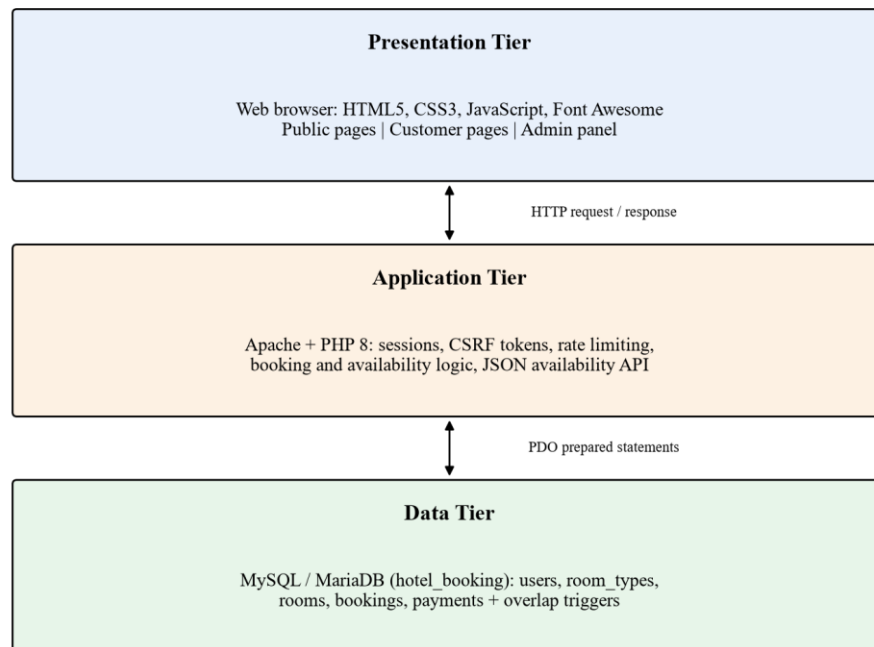


Figure 3.8: Three-Tier Architecture of StayEase

3.2.2 Database Schema Design

The ER diagram was converted into five relational tables, shown in Figure 3.9. Primary keys are auto-increment integers. Foreign keys link bookings to users and rooms, rooms to room types, and payments to bookings. Email addresses, room numbers and booking codes are unique. A check constraint ensures that the check-out date is after the check-in date, and composite indexes on room and dates speed up availability queries. The database script also defines triggers that reject overlapping bookings at the database level as an extra safeguard.

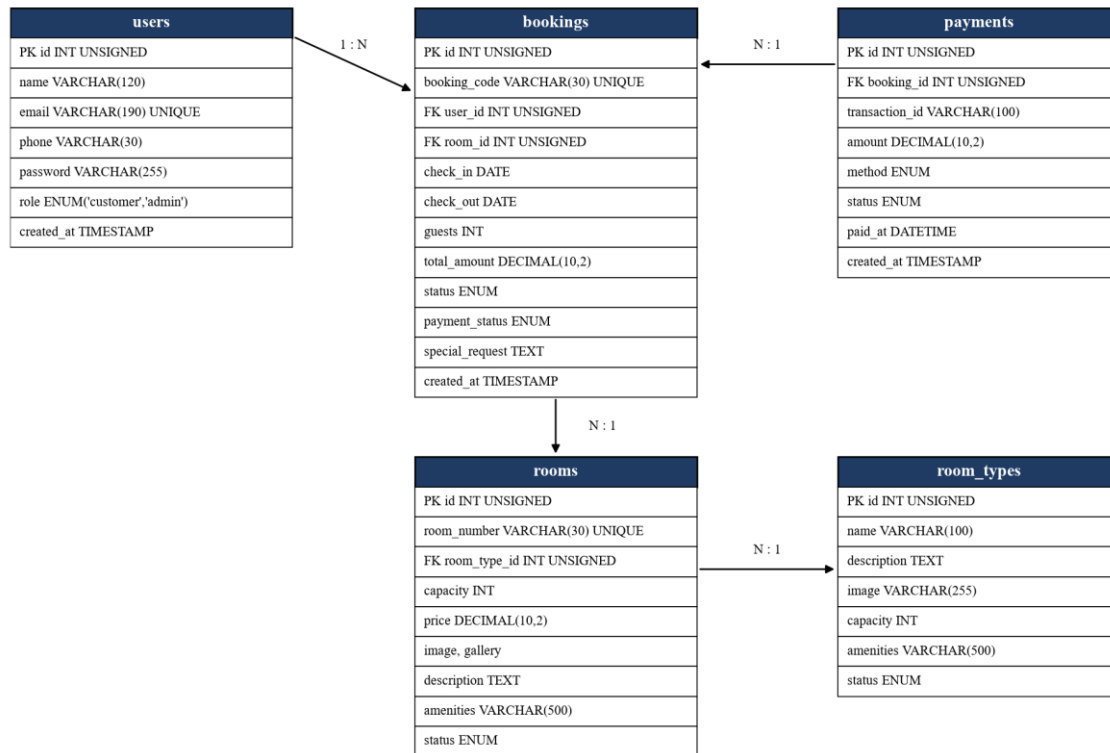


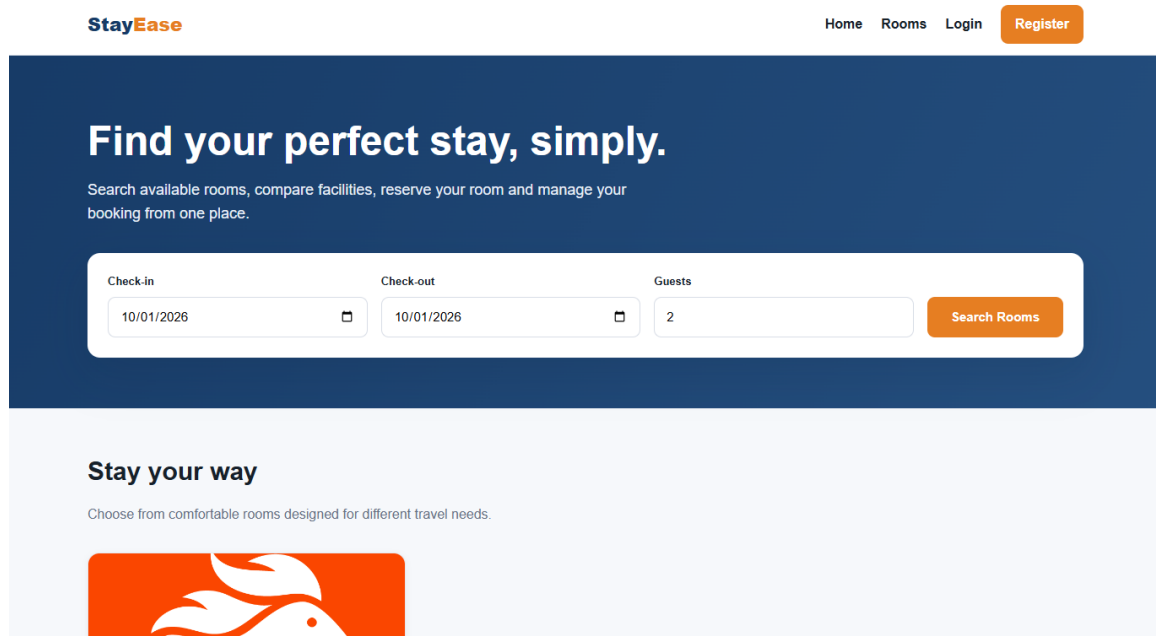
Figure 3.9: Database Schema

Table 3.2: Structure of the bookings Table

Field	Data Type	Constraint	Description
id	INT UNSIGNED	PK, auto increment	Unique booking identifier
booking_code	VARCHAR(30)	UNIQUE, NOT NULL	Code shown to the guest, e.g. BK260930A1B2C3
user_id	INT UNSIGNED	FK to users.id	Customer who made the booking
room_id	INT UNSIGNED	FK to rooms.id	Room that is reserved
check_in	DATE	NOT NULL	Arrival date
check_out	DATE	NOT NULL, > check_in	Departure date (exclusive)
guests	INT UNSIGNED	NOT NULL	Number of guests
total_amount	DECIMAL(10,2)	NOT NULL	Nights multiplied by nightly price
status	ENUM	default pending	pending, confirmed, cancelled, checked_in, checked_out
payment_status	ENUM	default pending	pending, paid, failed, refunded
special_request	TEXT	NULL	Contact details and guest request
created_at	TIMESTAMP	default now	Time the booking was made

3.2.3 Interface Design

The interface was designed to be simple and consistent. Public and customer pages share a top navigation bar with the StayEase logo and links to Home, Rooms, My Bookings and Login or Logout. The admin panel uses a dark sidebar with Dashboard, Bookings, Rooms, Room Types, Customers and Home links, and a content area with cards, tables and filters. Orange is used for primary actions and colour-coded badges show booking and payment status. Figures 3.10 to 3.13 show the main screens; more screens are included in the Appendix.



The screenshot displays the StayEase website's home page. At the top, a navigation bar includes the StayEase logo, links for Home, Rooms, Login, and a Register button. The main section features a large blue header with the text "Find your perfect stay, simply." and a subtext "Search available rooms, compare facilities, reserve your room and manage your booking from one place." Below this is a search form with fields for Check-in (10/01/2026), Check-out (10/01/2026), and Guests (2), accompanied by a Search Rooms button. The lower section, titled "Stay your way", includes the text "Choose from comfortable rooms designed for different travel needs." and a graphic of a stylized orange bird.

Figure 3.10: Home Page with Room Search

The screenshot shows the StayEase website interface. At the top, there's a navigation bar with 'Home', 'Rooms', 'My Bookings', and a 'Logout' button. Below the navigation bar, there's a 'Back to rooms' link. The main content area is split into two columns. The left column features a large image of the 'ICON FASHION' logo and a 'GALLERY' section with four small thumbnail images. The right column is titled 'Complete your booking' and contains a form with the following sections:

- Stay Dates:** Includes 'Check-in' (10/07/2026) and 'Check-out' (10/09/2026) date pickers.
- Duration:** Shows '2 night s' and a total price of 'NPR 9,000'.
- Number of guests:** A dropdown menu set to '2', with a note 'Maximum 2 guests'.
- Contact Information:** A section for entering contact details.

Figure 3.11: Room Details and Booking Form

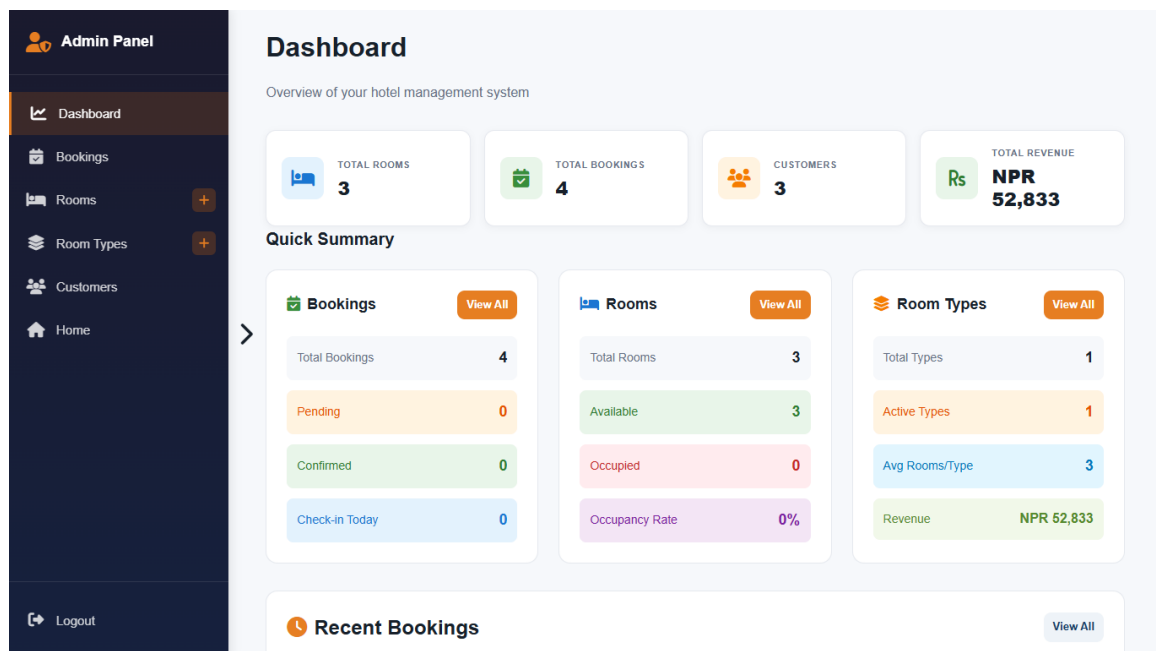


Figure 3.12: Admin Dashboard

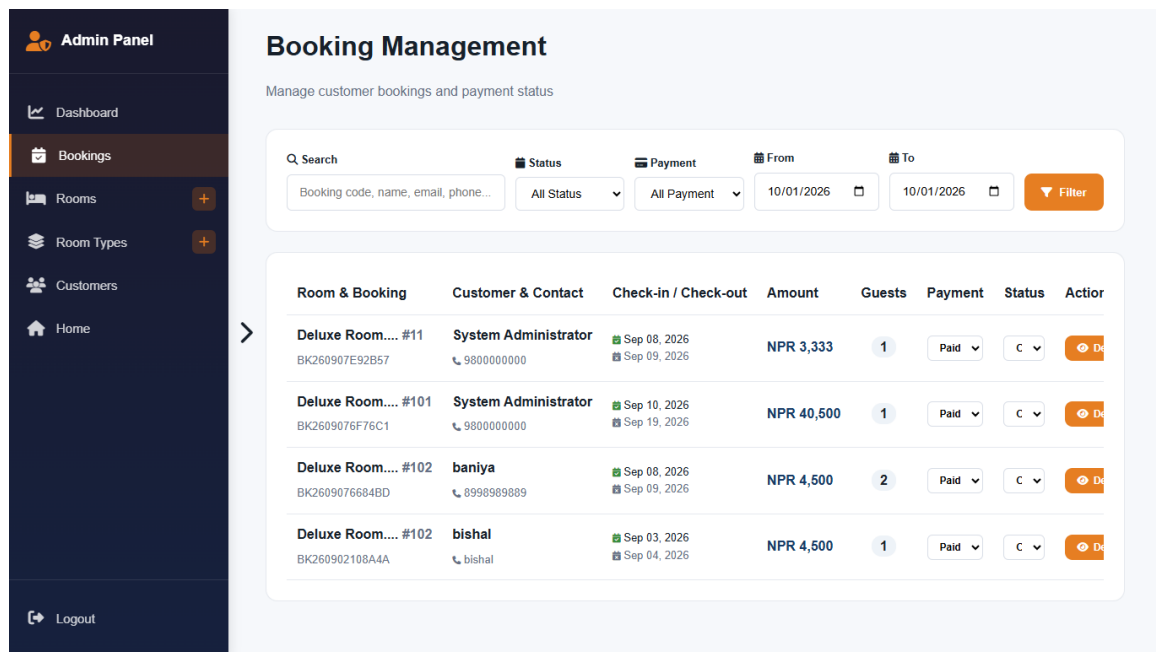


Figure 3.13: Admin Booking Management

3.2.4 Physical DFD

The physical DFD in Figure 3.14 shows how data actually moves through the implemented system. The user interacts with a web browser, which sends HTTP requests to the Apache server in XAMPP. Apache runs the relevant PHP script, such as book.php, payment.php or the admin pages, which reads and writes the MySQL database through PDO. Uploaded room images are stored in the uploads folder, and errors are written to a log file instead of being shown to the user. The generated HTML is returned to the browser.

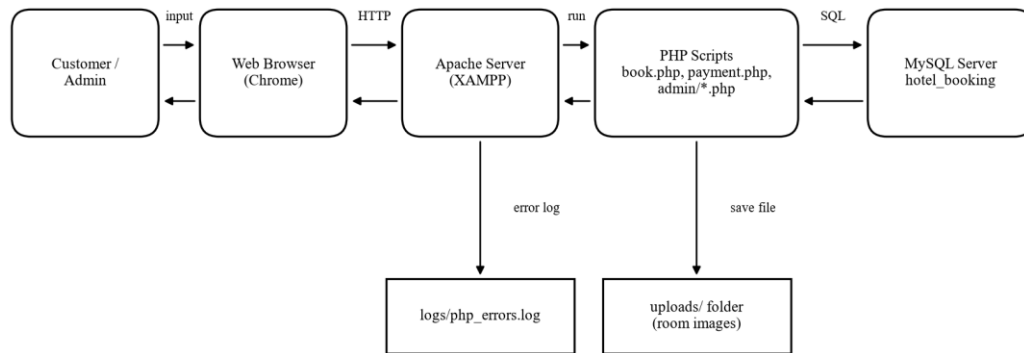


Figure 3.14: Physical DFD

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

StayEase was implemented as a PHP web application following the design in Chapter 3. Shared functions for database connection, escaping, redirects, flash messages, CSRF tokens, authentication, rate limiting and availability checking are kept in config/config.php and included by every page. Customer pages are in the project root, administrator pages are in the admin folder, and the availability API is in the api folder.

4.1.1 Tools Used

Programming Languages and Frameworks

- **HTML5:** Structure of all pages, forms and tables, including date inputs for check-in and check-out.
- **CSS3:** Styling with custom properties (CSS variables), flexbox and grid layouts, and responsive design for mobile screens.
- **JavaScript:** Client-side validation, live price and night calculation, image gallery, confirmation modals and toast notifications.
- **PHP 8:** Server-side logic, sessions, authentication, validation, booking transactions, payments and file uploads, using PDO for database access.
- **MySQL / MariaDB:** Relational database for users, room types, rooms, bookings and payments, with foreign keys, constraints, indexes and triggers.
- **Font Awesome 6.5:** Icon library used across the interface.

Tools and Software (CASE Tools)

- **XAMPP:** Local Apache web server, PHP runtime and MariaDB database server.
- **Visual Studio Code:** Code editor used for writing and organizing all source files.
- **phpMyAdmin:** Creating, importing and inspecting the hotel_booking database.
- **Google Chrome:** Running and testing the application, with developer tools for debugging.
- **Git and GitHub:** Version control of the source code.
- **draw.io / Python (Matplotlib):** Drawing the use case diagram, ER diagram, DFDs and architecture diagrams.

- **Microsoft Word:** Preparing the project report.

4.1.2 Implementation Details of Modules

- **Authentication Module (register.php, login.php, logout.php)**

Registration validates name length, email format, a 10-digit phone number and a password of at least six characters with an uppercase letter, a lowercase letter and a digit, then stores the password with `password_hash()` (bcrypt). Login verifies the password with `password_verify()`, regenerates the session ID, and stores the user in the session. The function `check_rate_limit()` blocks an IP and email pair after five failed attempts in 15 minutes. The functions `require_login()` and `require_admin()` protect customer and admin pages.

- **Room Search Module (index.php, rooms.php, api/availability.php)**

The home page lists active room types. The rooms page lists rooms with their type, capacity and price and filters them by dates, guests, maximum price and room type. When dates are given, each room is passed through `is_room_available()`, which returns true only if the room status is "available" and no pending, confirmed or checked-in booking satisfies `check_in < requested check-out` and `check_out > requested check-in`. The availability API returns the same result in JSON format.

- **Booking Module (room.php, book.php)**

The room page shows photos, amenities and a booking form with live calculation of nights and total price. On submission, `book.php` verifies the CSRF token and validates dates (not in the past, 1 to 30 nights), guests (within room capacity) and contact details. It then starts a transaction, locks the room row with `SELECT ... FOR UPDATE`, re-checks the room status and overlapping bookings, and rejects an identical booking made in the last five minutes. It generates a booking code in the form BK + date + six random hexadecimal characters, calculates the total as `nights()` multiplied by the nightly price, inserts the booking as pending, and commits. Any failure rolls back the transaction.

- **Payment Module (payment.php)**

The payment page shows the booking summary and total in NPR. On confirmation it checks that the booking belongs to the logged-in user and is still pending, generates a demo transaction ID, and in one transaction marks the payment as paid and the

booking as confirmed. The module is designed so that a real gateway such as eSewa or Khalti can replace the demo step.

- **My Bookings and Cancellation Module (my_bookings.php, cancel.php)**

Customers see all their bookings with code, room, dates, total, booking status and payment status. A booking can be cancelled only by its owner and only while it is pending or confirmed; the update query includes the user ID, which prevents one user from cancelling another user's booking.

- **Admin Dashboard Module (admin/index.php)**

The dashboard counts total rooms, bookings, customers, room types, pending and confirmed bookings, today's check-ins and available rooms, calculates occupied rooms and the occupancy rate, sums revenue from paid payments, and lists recent bookings.

- **Room Type and Room Management Module (admin/room_types.php, admin/rooms.php, admin/add_room.php)**

Administrators create, edit, enable, disable and delete room types and rooms. Room images and gallery photos are uploaded with checks on file size (maximum 5 MB) and real MIME type using finfo, and are saved with generated file names in the uploads folder. Rooms can be set to available, unavailable or maintenance; rooms that are not available are hidden from search and cannot be booked.

- **Booking Management Module (admin/bookings.php, admin/customers.php)**

Administrators view all bookings with customer and room details, filter them by status, update booking status to confirmed, checked-in, checked-out or cancelled, and update payment status. When a cash payment is marked as paid and no payment row exists, a new payment record is created. The customers page lists registered customers and their details.

- **Security Module (config/config.php)**

This module sets secure session cookies (HttpOnly, SameSite Strict), regenerates the session ID every 30 minutes, sends security headers including Content-Security-Policy and X-Frame-Options, logs errors to a file instead of displaying them, escapes all output with e(), and provides csrf_token() and verify_csrf() for every form.

4.2 Testing

StayEase was tested to make sure that each module works correctly on its own and that the modules work correctly together. Unit testing checked individual functions and pages with valid and invalid input, and system testing checked complete user journeys from registration to checkout.

4.2.1 Test Cases for Unit Testing

Table 4.1: Unit Test Cases for Registration

S.N.	Test Case	Input	Expected Result	Actual Result	Status
1	Register with valid data	Name, valid email, 10-digit phone, Demo@12345	Account created, redirect to login	Account created	Pass
2	Register with existing email	Email already in the database	Error: email already registered	Error shown	Pass
3	Weak password	Password "demo12"	Error: needs an uppercase letter	Error shown	Pass
4	Invalid phone number	Phone "98123"	Error: phone must be 10 digits	Error shown	Pass

Table 4.2: Unit Test Cases for Login

S.N.	Test Case	Input	Expected Result	Actual Result	Status
1	Login with valid admin credentials	admin@stayease.test / password	Redirect to admin dashboard	Dashboard displayed	Pass
2	Login with wrong password	Valid email / wrong password	Error: invalid credentials	Error shown	Pass
3	Invalid email format	"admin@"	Error: invalid email format	Error shown	Pass
4	Repeated failed login	6 wrong attempts in 15 minutes	Login temporarily blocked	Blocked with wait message	Pass
5	Customer opens admin page	Customer session, /admin/index.php	Access denied (403 Forbidden)	403 Forbidden shown	Pass

Table 4.3: Unit Test Cases for Room Search and Booking

S.N.	Test Case	Input	Expected Result	Actual Result	Status
1	Search available rooms	Future dates, 2 guests	Only free rooms with capacity ≥ 2 shown	Correct rooms listed	Pass
2	Check-out before check-in	Check-in 10th, check-out 8th	Error: check-out must be after check-in	Error shown	Pass
3	Past check-in date	Yesterday's date	Error: check-in	Error shown	Pass

S.N.	Test Case	Input	Expected Result	Actual Result	Status
			cannot be in the past		
4	Guests exceed capacity	5 guests for a 2-person room	Error: guests exceed room capacity	Error shown	Pass
5	Book an already booked room	Dates overlapping an active booking	Error: room no longer available	Booking rejected	Pass
6	Back-to-back booking	Check-in on the day another guest checks out	Booking accepted	Booking saved	Pass
7	Double click on Book	Same booking submitted twice	Second request rejected as duplicate	Duplicate message shown	Pass

Table 4.4: Unit Test Cases for Payment and Cancellation

S.N.	Test Case	Input	Expected Result	Actual Result	Status
1	Pay a pending booking	Click Pay & Confirm	Payment paid, booking confirmed	Status confirmed	Pass
2	Pay an already confirmed booking	Reopen payment page and submit	Error: already processed	Error shown	Pass
3	Cancel own pending booking	Click Cancel	Booking cancelled	Status cancelled	Pass
4	Cancel another user's booking	cancel.php?id=<other user's booking>	Error: cannot be cancelled	Error shown	Pass

Table 4.5: Unit Test Cases for Admin Room Management

S.N.	Test Case	Input	Expected Result	Actual Result	Status
1	Add room type	Name, description, image	Room type added and listed	Added	Pass
2	Add room with duplicate number	Existing room number	Error: room number exists	Error shown	Pass
3	Upload non-image file	file.pdf renamed as .jpg	Error: invalid file type	Upload rejected	Pass
4	Set room to maintenance	Status = maintenance	Room hidden from search and cannot be booked	Room hidden	Pass

4.2.2 Test Cases for System Testing

Table 4.6: System Test Case for the Complete Booking Journey

S.N.	Test Step	Expected Result	Actual Result	Status
1	Open home page and search rooms for future dates	Available rooms listed with price	Rooms listed	Pass

S.N.	Test Step	Expected Result	Actual Result	Status
2	Register a new customer and log in	Customer logged in, navigation shows My Bookings	Logged in	Pass
3	Open a room, enter details and book	Booking saved as pending with a booking code	Booking code shown	Pass
4	Pay for the booking	Booking confirmed, payment paid	Confirmed	Pass
5	Admin logs in and opens Bookings	New booking appears with status confirmed	Booking visible	Pass
6	Admin marks booking as checked-in, then checked-out	Status updated at each step	Status updated	Pass
7	Admin dashboard is refreshed	Booking count and revenue updated	Updated	Pass
8	Second customer searches the same dates	Booked room no longer offered for those dates	Room not listed	Pass
9	Customer logs out	Session ended, redirect to home page	Logged out	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Lesson Learnt / Outcome

Developing StayEase gave practical experience of the complete software development life cycle, from requirement collection and feasibility study through design, coding, testing and documentation. The most important technical lesson was that a feature which looks simple, such as checking whether a room is free, becomes difficult when several users act at the same time. Solving it required an understanding of date-range overlap, database transactions and row-level locking. The project also showed the value of designing the database carefully first, because clear tables, keys and constraints made every later module easier to build.

Security was another major lesson. Implementing password hashing, prepared statements, CSRF tokens, output escaping, rate limiting, secure file uploads and ownership checks showed how common attacks work and how they are prevented. Working in iterations with regular supervisor feedback improved planning, time management, debugging and report writing skills.

The outcomes of the project are:

- Customers can register, log in, search available rooms, view details, book, pay (demo), view and cancel bookings.
- Administrators can manage room types, rooms, images, room status, bookings, payment status and customers.
- The system prevents double booking, including under simultaneous requests.
- The dashboard gives the hotel a real-time view of bookings, occupancy and revenue.
- The system is protected against the most common web application attacks.

5.2 Conclusion

The StayEase Online Hotel Booking and Management System meets the objectives set at the start of the project. It replaces manual, register-based reservation with a single online system in which guests can find and book rooms at any time and hotel staff can manage rooms, bookings and payments from one panel. Automatic night and price calculation,

unique booking codes and a clear booking life cycle reduce errors and disputes, while the availability engine with transaction-level locking ensures that a room is never booked twice for the same night. Although it is an academic project with a demo payment step and a single-hotel scope, StayEase is a working and secure base that a small hotel could extend and deploy.

5.3 Future Recommendations

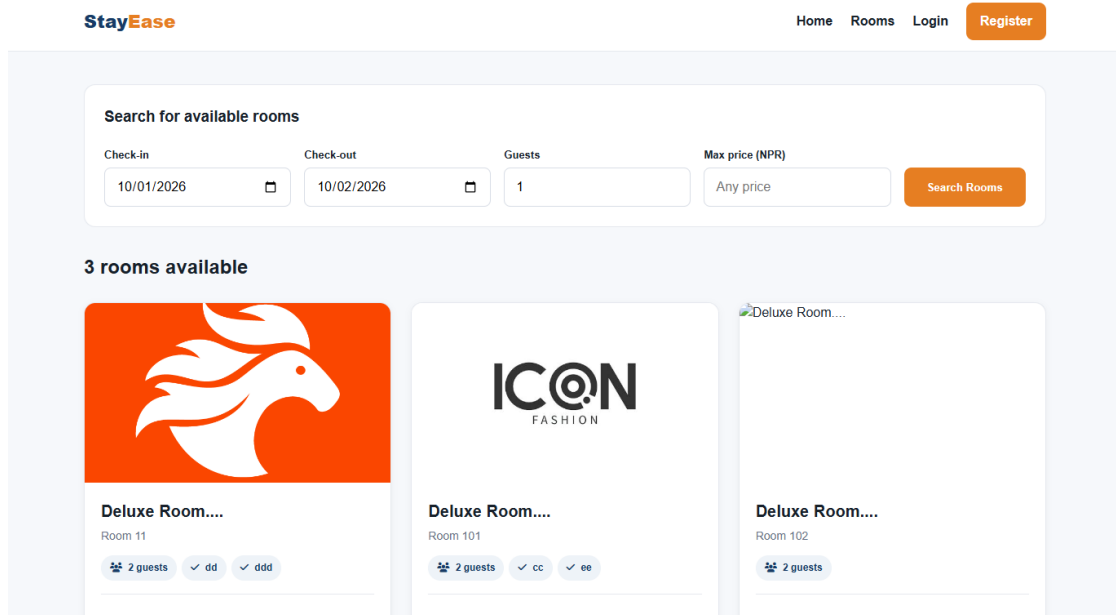
- Integrate real payment gateways used in Nepal, such as eSewa, Khalti and Fonepay, with refund handling.
- Send booking confirmation, reminder and cancellation messages by email and SMS.
- Add separate roles for receptionist, housekeeping and accountant with their own permissions.
- Add seasonal and weekend pricing, discount coupons and loyalty points.
- Allow guests to rate and review rooms after checkout.
- Generate downloadable invoices and monthly occupancy and revenue reports with charts.
- Support multiple hotel branches and synchronize availability with OTAs through a channel manager.
- Develop a mobile application using the existing availability API.

REFERENCES

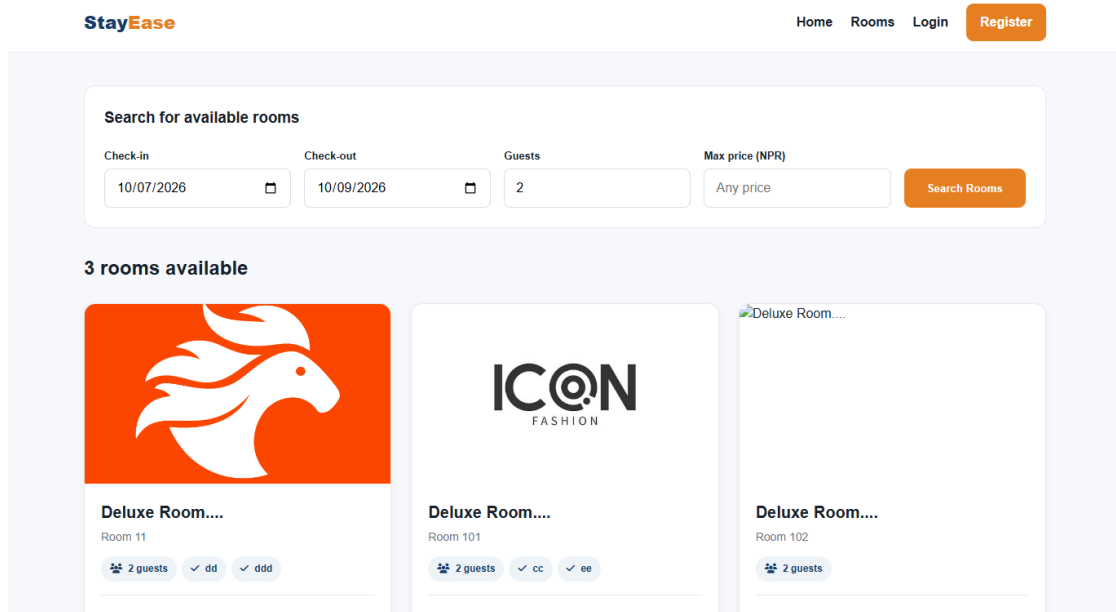
- [1] D. Buhalis and R. Law, "Progress in information technology and tourism management: 20 years on and 10 years after the Internet - The state of eTourism research," *Tourism Management*, vol. 29, no. 4, pp. 609-623, 2008.
- [2] P. O'Connor and A. J. Frew, "The future of hotel electronic distribution: Expert and industry perspectives," *Cornell Hotel and Restaurant Administration Quarterly*, vol. 43, no. 3, pp. 33-45, 2002.
- [3] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
- [5] Oracle Corporation, "MySQL 8.0 Reference Manual: InnoDB Locking Reads," [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>
- [6] OWASP Foundation, "OWASP Top 10:2021," [Online]. Available: <https://owasp.org/Top10/>
- [7] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [8] OWASP Foundation, "Password Storage Cheat Sheet," [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [9] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [10] The PHP Group, "PHP Manual: PHP Data Objects," [Online]. Available: <https://www.php.net/manual/en/book.pdo.php>
- [11] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: <https://agilemanifesto.org/>

APPENDICES

Appendix A: Screenshots



Screenshot: Room Listing Page



Screenshot: Room Search Results by Date and Guests

StayEase

HomeRoomsLoginRegister

StayEase

Welcome back! Sign in to manage your bookings and discover great stays.

 **Browse Hotels**
Find the perfect room for your stay

 **Easy Booking**
Reserve rooms in just a few clicks

 **Manage Bookings**
Track and manage your reservations

Welcome back

Sign in to your account to continue

Email address

you@example.com

Password

Enter your password

Sign In

Demo Credentials

Admin: admin@stayease.test / password

Don't have an account? [Create one](#)

Screenshot: Login Page

StayEase

HomeRoomsLoginRegister

StayEase

Join us today and start enjoying seamless hotel bookings and exceptional service.

 **Quick Registration**
Create your account in seconds

 **Secure & Private**
Your data is safe with us

 **Personalized Experience**
Track bookings and preferences

Create account

Sign up to start booking your perfect stay

Full name

John Doe

Minimum 3 characters

Email address

you@example.com

We'll never share your email

Phone number (Optional)

9800000000

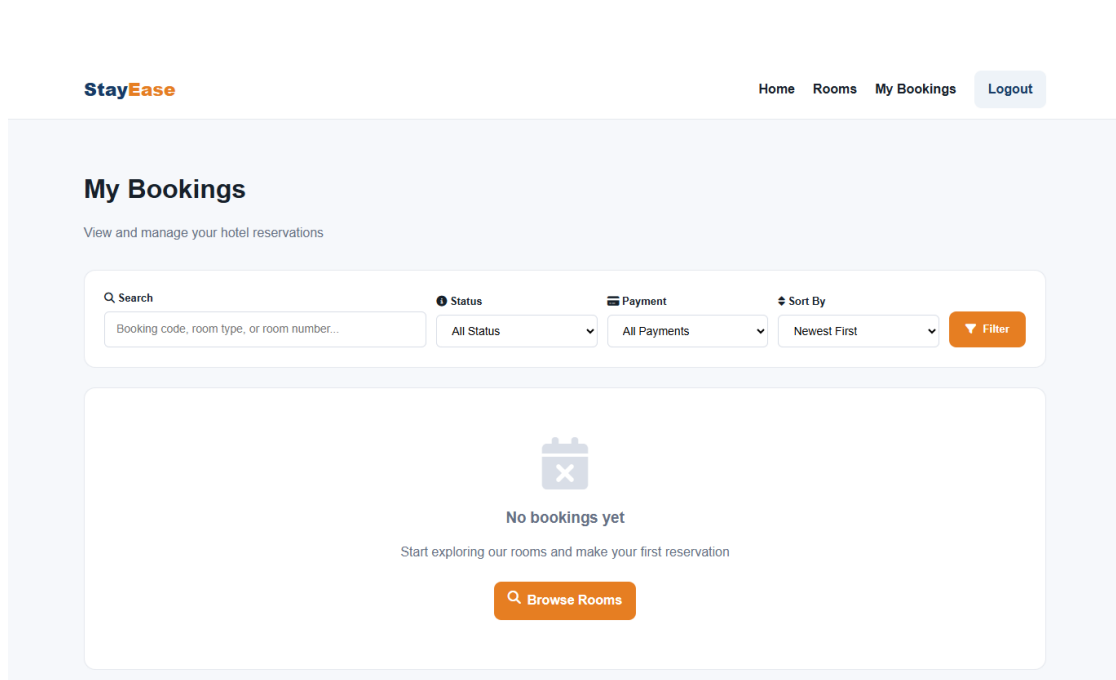
10 digit number

Password

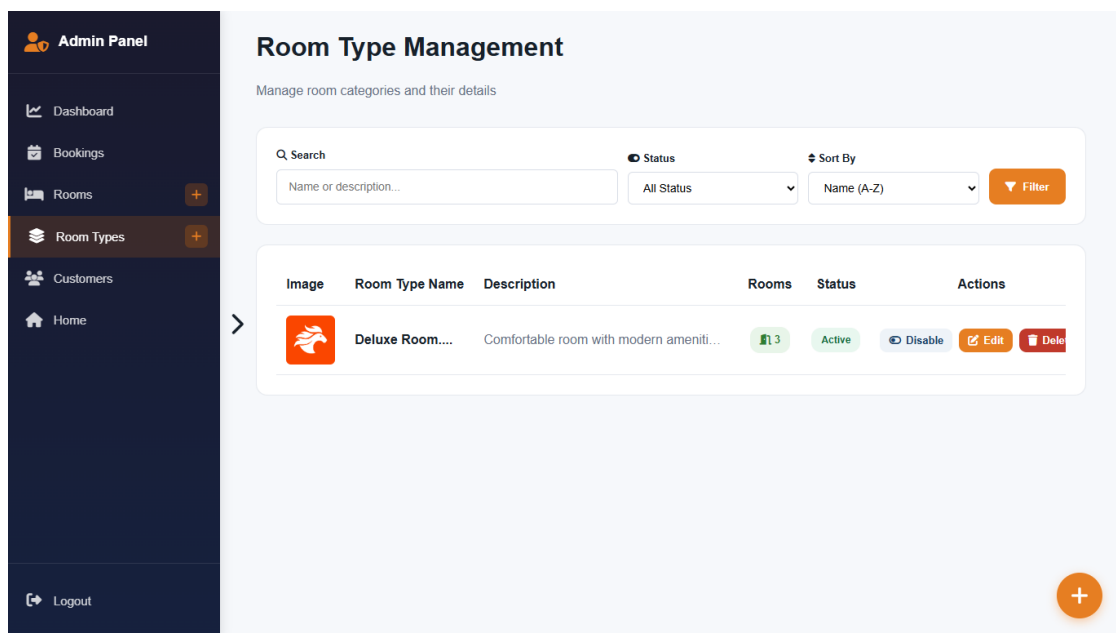
Create a strong password

Must contain uppercase, lowercase, and number

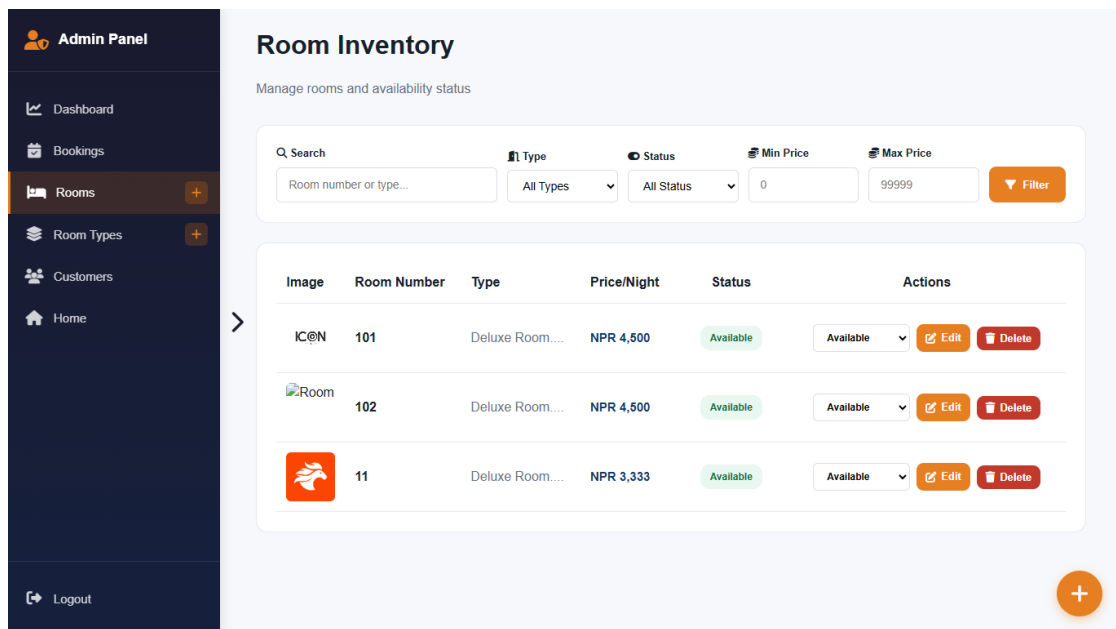
Screenshot: Customer Registration Page



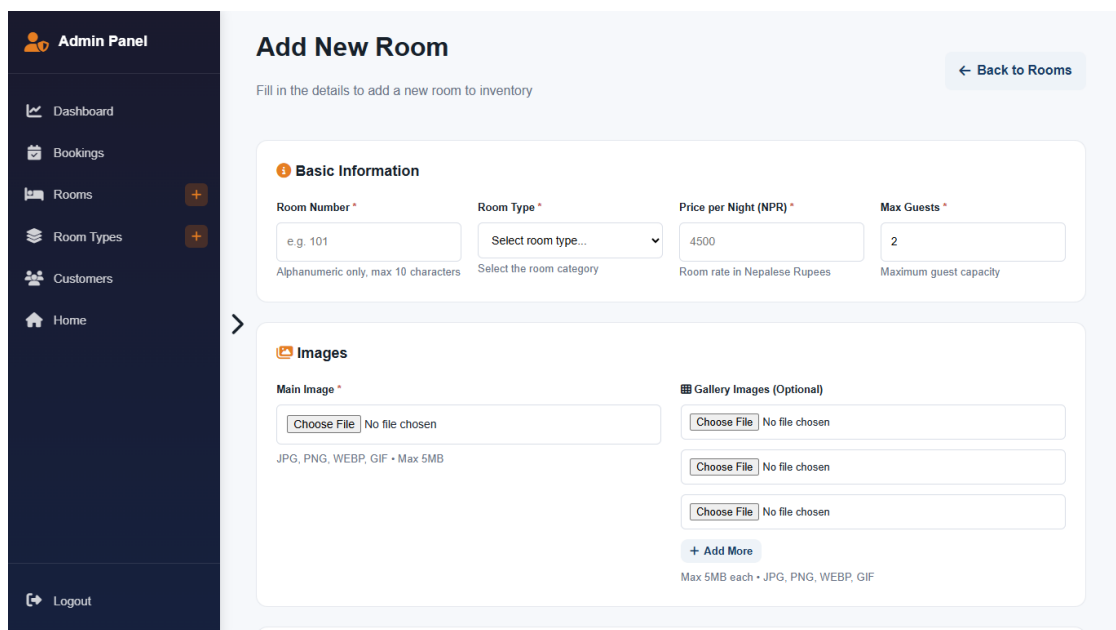
Screenshot: My Bookings Page



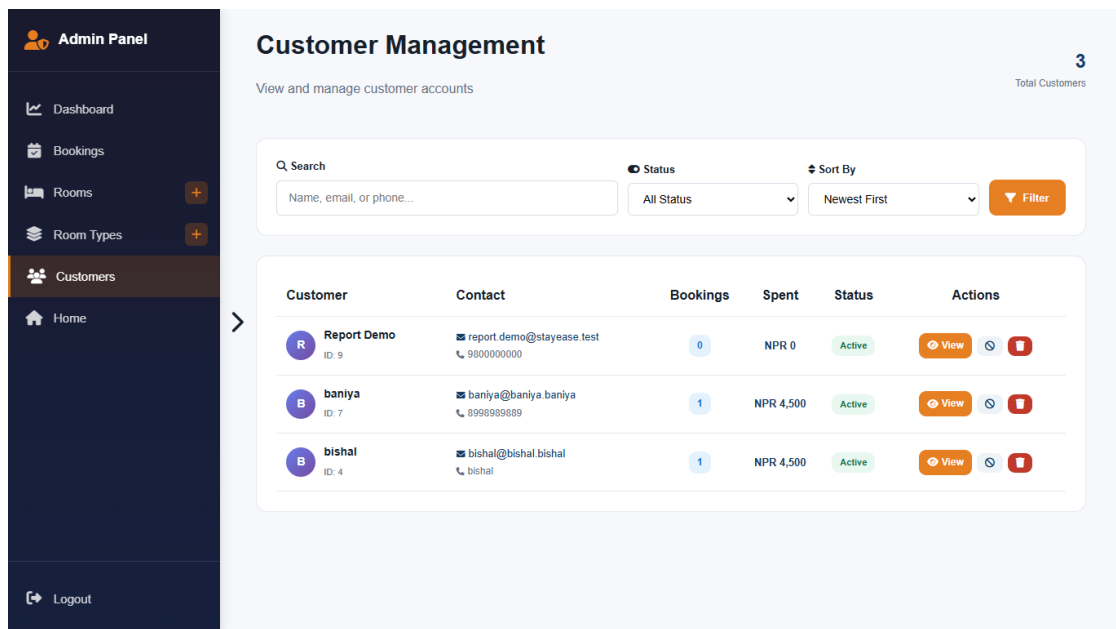
Screenshot: Admin Room Type Management



Screenshot: Admin Room Management



Screenshot: Admin Add Room Form



Screenshot: Admin Customer List

Appendix B: Source Code Excerpt

The function below, from config/config.php, decides whether a room is free for a given stay.

```
function is_room_available(int $roomId, string $checkin, string $checkout, ?int $ignoreBookingId = null): bool {
    // First check if room itself is available (not unavailable or in maintenance)
    $roomCheck = db()->prepare("SELECT status FROM rooms WHERE id = ?");
    $roomCheck->execute([$roomId]);
    $roomStatus = $roomCheck->fetchColumn();

    if($roomStatus !== 'available') {
        return false;
    }

    // Then check for booking conflicts
    $sql = "SELECT COUNT(*) FROM bookings
        WHERE room_id = ?
            AND status IN ('pending','confirmed','checked_in')
            AND check_in < ?
            AND check_out > ?";
    $params = [$roomId, $checkout, $checkin];
    if ($ignoreBookingId) {
        $sql .= " AND id <> ?";
        $params[] = $ignoreBookingId;
    }
    $stmt = db()->prepare($sql);
    $stmt->execute($params);
    return (int)$stmt->fetchColumn() === 0;
}
```

Appendix C: Supervisor Visit Log Sheet

Table C.1: Supervisor Visit Log Sheet

S.N.	Date	Work Reviewed / Discussion	Supervisor Remarks	Signature
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				